

AI Assisted Coding

Assignment-17

NAME : K.SAI TEJA

HT NO :2303A52325

BATCH :35

Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts

Objective

- To clean raw datasets using Python and AI-assisted scripting.
- To handle missing values, remove duplicates, and standardize data.
- To encode categorical variables and normalize numerical data.
- To automate preprocessing tasks for efficient data analysis.
- To understand the importance of preprocessing in improving model performance.

Task 1 – Customer Data Cleaning

Aim

To clean and preprocess a customer shopping dataset.

Procedure

- Loaded dataset using Pandas.
- Handled missing values in Age, Income, and City columns.
- Removed duplicate records.
- Standardized city names to lowercase.
- Encoded categorical variables such as Gender and City.

Code

```
import pandas as pd
```

```
df = pd.read_csv("shopping_trends.csv")
```

```
df['Age'] = df['Age'].fillna(df['Age'].median())
```

```
df['Purchase Amount (USD)'] = df['Purchase Amount (USD)'].fillna(df['Purchase Amount (USD)'].mean())
```

```
df['Location'] = df['Location'].fillna(df['Location'].mode()[0])
```

```
df = df.drop_duplicates()
```

```
df['Location'] = df['Location'].str.lower().str.strip()
```

```
df = pd.get_dummies(df, columns=['Gender', 'Location'], drop_first=True)
```

```
print(df.head())
```

Result

A cleaned dataset with no missing values, standardized text, and encoded categorical features.

```
import pandas as pd

# Load dataset
df = pd.read_csv("../content/shopping_trends_updated.csv")

# Handle missing values
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Purchase Amount (USD)'] = df['Purchase Amount (USD)'].fillna(df['Purchase Amount (USD)'].mean())
df['Location'] = df['Location'].fillna(df['Location'].mode()[0])

# Remove duplicates
df = df.drop_duplicates()

# Standardize text (city/location)
df['Location'] = df['Location'].str.lower().str.strip()

# Encode categorical variables
df = pd.get_dummies(df, columns=['Gender', 'Location'], drop_first=True)

print(df.head())
```

Customer ID	Age	Item Purchased	Category	Purchase Amount (USD)	Size	\
0	1	25	Blouse	Clothing	53	L
1	2	19	Sweater	Clothing	64	L
2	3	50	Jeans	Clothing	73	S
3	4	21	Sandals	Footwear	90	M
4	5	45	Blouse	Clothing	49	M

Color	Season	Review Rating	Subscription Status	...	\
Gray	Winter	3.1	Yes	...	...
Maroon	Winter	3.1	Yes	...	...
Maroon	Spring	3.1	Yes	...	...
Maroon	Spring	3.5	Yes	...	...
Turquoise	Spring	2.7	Yes	...	...

Location_south_dakota	Location_tennessee	Location_texas	Location_utah	\
False	False	False	False	...
False	False	False	False	...
False	False	False	False	...
False	False	False	False	...
False	False	False	False	...

Location_vermont	Location_virginia	Location_washington	\
False	False	False	...
False	False	False	...
False	False	False	...
False	False	False	...
False	False	False	...

Location_west_virginia	Location_wisconsin	Location_wyoming	\
False	False	False	...
False	False	False	...
False	False	False	...
False	False	False	...
False	False	False	...

[5 rows x 66 columns]

Task 2 – Sales Data Preprocessing

Aim

To preprocess sales transaction data and create useful features.

Procedure

- Converted date column into datetime format.
- Extracted month, year, and day of the week.
- Normalized the sales column using Min-Max scaling.

- Detected and removed outliers using the IQR method.

Code

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv("sales_data.csv")

df['date'] = pd.to_datetime(df['date'])

df['month'] = df['date'].dt.month
df['year'] = df['date'].dt.year
df['day_of_week'] = df['date'].dt.dayofweek

scaler = MinMaxScaler()
df['sales'] = scaler.fit_transform(df[['sales']])

Q1 = df['sales'].quantile(0.25)
Q3 = df['sales'].quantile(0.75)
IQR = Q3 - Q1

df = df[(df['sales'] >= Q1 - 1.5*IQR) & (df['sales'] <= Q3 + 1.5*IQR)]

print(df.head())
```

Result

A transformed dataset with normalized values and additional time-based features.

```

import pandas as pd
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv("/content/sales_data.csv")

# Convert date
df['Sale Date'] = pd.to_datetime(df['Sale Date'])

# Feature engineering
df['month'] = df['Sale Date'].dt.month
df['year'] = df['Sale Date'].dt.year
df['day_of_week'] = df['Sale Date'].dt.dayofweek

# Normalize sales
scaler = MinMaxScaler()
df['sales'] = scaler.fit_transform(df[['Sales Amount']])

# Outlier removal (IQR method)
Q1 = df['sales'].quantile(0.25)
Q3 = df['sales'].quantile(0.75)
IQR = Q3 - Q1

df = df[(df['sales'] >= Q1 - 1.5*IQR) & (df['sales'] <= Q3 + 1.5*IQR)]

print(df.head())

```

	Product_ID	Sale Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	\
0	1052	2023-03-03	Bob	North	5853.97	18	
1	1093	2023-04-21	Bob	West	4386.02	17	
2	1015	2023-09-21	David	South	4631.23	30	
3	1072	2023-08-24	Bob	South	2187.94	39	
4	1061	2023-03-24	Charlie	East	3750.20	13	

	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	\
0	Furniture	152.75	267.22	Returning	0.09	
1	Furniture	3816.30	4289.44	Returning	0.11	
2	Food	261.56	371.68	Returning	0.20	
3	Clothing	4330.03	4667.75	New	0.02	
4	Electronics	637.37	692.71	New	0.08	

	Payment_Method	Sales_Channel	Region_and_Sales_Rep	month	year	day_of_week	\
0	Cash	Online	North-Bob	2	2023	4	
1	Cash	Retail	West-Bob	4	2023	4	
2	Bank Transfer	Retail	South-David	9	2023	3	
3	Credit Card	Retail	South-Bob	8	2023	3	
4	Credit Card	Online	East-Charlie	3	2023	4	

	sales
0	0.508950
1	0.433202
2	0.458201
3	0.200105
4	0.300108

Task 3 – Healthcare Data Preparation

Aim

To preprocess healthcare data for machine learning.

Procedure

- Filled missing values in blood pressure and cholesterol.
- Encoded categorical variables using Label Encoding.
- Scaled numerical features using Standard Scaling.
- Split dataset into training and testing sets.

Code

```

import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split

df = pd.read_csv("healthcare_dataset.csv")

df['blood_pressure'] = df['blood_pressure'].fillna(df['blood_pressure'].mean())
df['cholesterol'] = df['cholesterol'].fillna(df['cholesterol'].mean())

le = LabelEncoder()

```

```
for col in df.select_dtypes(include='object').columns:
```

```
    df[col] = le.fit_transform(df[col])
```

```
scaler = StandardScaler()
```

```
df[['blood_pressure', 'cholesterol']] = scaler.fit_transform(df[['blood_pressure', 'cholesterol']])
```

```
X = df.drop('target', axis=1)
```

```
y = df['target']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
print(X_train.shape, X_test.shape)
```

Result

A fully preprocessed dataset split into training and testing sets, ready for model training.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split

df = pd.read_csv("/content/dirty_v3_path.csv")

# Handle missing values
df['Blood Pressure'] = df['Blood Pressure'].fillna(df['Blood Pressure'].mean())
df['Cholesterol'] = df['Cholesterol'].fillna(df['Cholesterol'].mean())

# Encode categorical
le = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

# Scale numerical features
scaler = StandardScaler()
num_cols = ['Blood Pressure', 'Cholesterol']
df[num_cols] = scaler.fit_transform(df[num_cols])

# Split data
X = df.drop('Medical Condition', axis=1)
y = df['Medical Condition']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

print(X_train.shape, X_test.shape)

(20000, 10) (5000, 10)
```

Task 4 – Employee Salary Data Preprocessing

Aim

To clean and preprocess employee salary data for analysis.

Procedure

- Handled missing values using forward fill.
- Removed salary outliers using the IQR method.
- Standardized department names to lowercase.
- Encoded job location and department using Label Encoding.

Code

```
import pandas as pd
```

```
from sklearn.preprocessing import LabelEncoder
```

```
df = pd.read_csv("employee_salary.csv")
```

```
df = df.fillna(method='ffill')
```

```
Q1 = df['salary'].quantile(0.25)
```

```
Q3 = df['salary'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
df = df[(df['salary'] >= Q1 - 1.5*IQR) & (df['salary'] <= Q3 + 1.5*IQR)]
```

```
df['department'] = df['department'].str.lower().str.strip()
```

```
le = LabelEncoder()
```

```
df['job_location'] = le.fit_transform(df['job_location'])
```

```
df['department'] = le.fit_transform(df['department'])
```

```
print(df.head())
```

Result

A structured dataset with cleaned and encoded features suitable for HR analytics.

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv("../content/sales_data.csv")

# Convert date
df['Sale_Date'] = pd.to_datetime(df['Sale_Date'])

# Feature engineering
df['month'] = df['Sale_Date'].dt.month
df['year'] = df['Sale_Date'].dt.year
df['day_of_week'] = df['Sale_Date'].dt.dayofweek

# Normalize sales
scaler = MinMaxScaler()
df['sales'] = scaler.fit_transform(df[['Sales_Amount']])

# Outlier removal (IQR method)
Q1 = df['sales'].quantile(0.25)
Q3 = df['sales'].quantile(0.75)
IQR = Q3 - Q1

df = df[(df['sales'] >= Q1 - 1.5*IQR) & (df['sales'] <= Q3 + 1.5*IQR)]

print(df.head())
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	\
0	1852	2023-02-03	Bob	North	5853.97	18	
1	1803	2023-04-21	Bob	West	4384.82	17	
2	1815	2023-09-21	David	South	4631.23	38	
3	1872	2023-08-24	Bob	South	2167.94	39	
4	1861	2023-01-24	Charlie	East	3758.20	13	

	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	\
0	Furniture	152.75	267.22	Returning	0.09	
1	Furniture	3116.20	4209.44	Returning	0.11	
2	Food	261.56	371.68	Returning	0.28	
3	Clothing	4338.83	4467.75	New	0.82	
4	Electronics	637.37	692.71	New	0.08	

	Payment_Method	Sales_Channel	Region_and_Sales_Rep	month	year	day_of_week	\
0	Cash	Online	North-Bob	2	2023	4	
1	Cash	Retail	West-Bob	4	2023	4	
2	Bank Transfer	Retail	South-David	9	2023	3	
3	Credit Card	Retail	South-Bob	8	2023	3	
4	Credit Card	Online	East-Charlie	3	2023	4	

	sales
0	0.508958
1	0.433282
2	0.458281
3	0.389185
4	0.389188

Conclusion

Data preprocessing is a crucial step in machine learning as it ensures data quality, consistency, and reliability. Proper handling of missing values, encoding, normalization, and outlier removal significantly improves model accuracy and performance.

